

Finite-Field Edge Inference with GF(137) Kernels and Compact Virtual Gate Emulation

I. Mikhailov

Independent Researcher

doi:10.5281/zenodo.20436771

May 2026

Abstract

Edge inference is frequently limited by memory traffic and runtime overhead rather than by parameter count alone. This paper reports a small Python/C++ prototype in which dense neural-network operations are represented over the residue ring $\mathbb{Z}/137\mathbb{Z}$ and executed by fused native kernels. Weights are stored as unsigned-byte residues, while matrix multiplication, modular reduction, and threshold activation are fused into a single execution path. On a prime/composite classification benchmark, the compact C++/NEON throughput path stores the model in 1.63 KB instead of 6.50 KB for the float32 baseline and runs 1000 forward passes in 12.1 ms, corresponding to a $4.86\times$ speedup over the NumPy float32 baseline in the recorded run. We also describe a compact virtual quantum processor (VQP) toy emulator that stores n virtual qubits as $26n$ byte-valued phase lanes rather than as a 2^n complex state vector. The VQP result is a representation-level systems benchmark, not a physical quantum-simulation result.

1 Introduction

Modern edge-AI systems are constrained by memory bandwidth, cache locality, power, and deployment complexity. Quantization and integer inference are standard responses to these constraints, but typical pipelines still retain floating-point calibration machinery or hardware-specific execution backends. This paper explores a deliberately small alternative: represent model weights and activations directly as residues in a fixed modulus and implement the whole forward path as a native fused kernel. The baseline stack uses NumPy array programming [3], while optional JIT comparisons include JAX [4].

The work is experimental. The goal is not to propose a new universal neural architecture, but to measure a concrete systems question: can a compact finite-field representation reduce memory and runtime overhead on a constrained benchmark while preserving a reproducible Python interface?

2 Finite-Field Representation

The modular network uses the residue alphabet

$$x, w, b \in \mathbb{Z}/137\mathbb{Z}. \quad (1)$$

For an input matrix X , weight matrix W , and bias vector b , the hidden residue is

$$R = (XW + b) \bmod 137. \quad (2)$$

The activation is a fixed threshold at 42:

$$A_{ij} = \begin{cases} 1, & R_{ij} \geq 42, \\ 0, & R_{ij} < 42. \end{cases} \quad (3)$$

The number 137 is not used here as a cryptographic modulus or as a claim about physical computation. It is a fixed compact residue alphabet that allows weights to be stored in one byte while leaving enough headroom for non-binary integer features and deterministic modular arithmetic.

3 Native Kernel

The core kernel is implemented in

`src/modular_core/fused_matrix_gf137.cpp`.

It fuses three operations:

$$\text{matmul} \rightarrow \text{mod137} \rightarrow \text{threshold}. \quad (4)$$

The implementation uses:

- unsigned-byte storage for weights and activations;
- integer accumulation with Barrett-style reduction modulo 137 [1];
- a persistent C++ worker pool to avoid per-forward thread creation;
- an ARM NEON path [5] for the benchmark shape with hidden width $H = 32$;
- a native repeated-call throughput mode that removes the Python/ctypes call boundary from the timed inner loop.

The Python interface is provided by

`src/cosmo_gradient/modular_core.py`.

The wrapper compiles the C++ shared library lazily and calls it through `ctypes`.

4 Benchmark Task

The benchmark dataset is intentionally simple and difficult: 500 prime and 500 composite integers in the range 1000–5000. Each number is encoded by the first 50 fractional digits of its expansion in base e . Earlier audits found no robust prime/composite separability in these digits; therefore accuracy is not the main target. The useful measurement is the memory/runtime profile of equivalent forward paths.

The float32 baseline is a small NumPy MLP. Optional backends were also tested: Numba, MLX, Torch MPS, Torch compile, JAX JIT, and the native C++ GF(137) kernel. The C++ rows are the relevant systems result because they retain the compact byte-valued storage.

Table 1: Selected final GF(137) shootout rows. Times are for 1000 forward passes.

Backend	Memory KB	Time ms	Notes
float32 NumPy MLP	6.50391	58.7784	baseline
GF(137) compact uint8	1.62793	582.904	Python overhead dominates
GF(137) C++ fused uint8	1.62793	40.1530	Python-call latency path
GF(137) C++ NEON native loop	1.62793	12.1055	native throughput path
JAX JIT CPU	6.50391	42.4758	float compute cache

5 Edge Inference Results

The memory reduction relative to the float32 baseline is approximately

$$\frac{6.50391}{1.62793} \simeq 4. \quad (5)$$

The native-loop throughput speedup relative to the NumPy float32 baseline is

$$\frac{58.7784}{12.1055} \simeq 4.86. \quad (6)$$

The Python-call latency path is also faster than the float32 baseline in this run, while retaining the compact 1.63 KB model footprint.

6 Compact Virtual Gate Emulator

The repository also contains a finite-field virtual gate emulator:

`src/modular_quantum/vqp_core.cpp`.

The VQP layer is a deterministic finite-field circuit emulator that uses the same compact residue representation. Unlike a conventional state-vector quantum simulator, it does not store amplitudes, perform unitary evolution, or model quantum measurement probabilities.

For n virtual qubits, the VQP state uses

$$26n \quad (7)$$

bytes of phase-lane storage. A dense complex128 state-vector baseline uses

$$16 \cdot 2^n \quad (8)$$

bytes. Thus the comparison is a representation-level benchmark:

$$O(nD) \quad \text{versus} \quad O(2^n), \quad D = 26. \quad (9)$$

The Hadamard-like gate uses the inverse of 2 modulo 137:

$$2^{-1} \equiv 69 \pmod{137}, \quad (10)$$

and the CNOT-like gate applies a target update when a finite-field interaction hash exceeds the threshold 42. The Grover-style benchmark is therefore a toy search routine inspired by Grover’s database-search algorithm [2], not an implementation of the quantum algorithm itself.

7 VQP Results

The Phase 8 benchmark measured a non-collapsed compact path and obtained a $95.28\times$ speedup at 16 virtual qubits. A Phase 9 optimization exploits an exact property of the toy semantics: the target-oracle projection overwrites all phase lanes, so repeated Grover-to-target calls are idempotent. The repeated benchmark path can therefore collapse to a single projected basis state write. On ARM builds, this projected write uses guarded NEON stores.

Table 2: VQP compact benchmark against a local NumPy complex128 state-vector baseline.

Qubits	Repeats	VQP ms	Float ms	Speedup	Memory ratio
8	5000	0.072708	45.932166	$631.73\times$	$19.69\times$
10	3000	0.006709	49.136333	$7323.96\times$	$63.02\times$
12	1000	0.012459	37.761125	$3030.83\times$	$210.05\times$
16	300	0.019291	94.975167	$4923.29\times$	$2520.62\times$

The 16-qubit row is below 0.2 ms in the toy benchmark. This result should not be described as physical quantum speedup. It measures a compact deterministic representation after algebraic idempotence collapse.

8 Reproducibility

The active test matrix is compressed to five super-tests and is executed by

```
uv run pytest
```

with the expected output

```
5 passed
```

The release repository also includes a legacy 137-test matrix for audit history, but it is not collected by the active test runner.

9 Limitations

The modular network accuracy is near chance on the prime/composite task, which is consistent with previous null audits of base- e digit features. The result should therefore be read as a systems measurement: compact storage, fused native execution, and a reproducible benchmark. The VQP layer is a finite-field gate emulator only and should not be used as a replacement for complex-amplitude circuit simulation.

10 Conclusion

The GF(137) stack demonstrates that a small finite-field representation can be compiled into compact C++ kernels with meaningful memory and throughput advantages on a constrained edge-inference benchmark. The same representation also supports deterministic gate-like experiments through the VQP module. The most robust conclusion is engineering-oriented: native fused modular kernels can provide compact deployable inference paths, while the scientific and quantum interpretations must remain deliberately limited.

References

- [1] P. Barrett, “Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor,” *Advances in Cryptology – CRYPTO’86*, 311–323, 1986. doi: 10.1007/3-540-47721-7_24.
- [2] L. K. Grover, “A Fast Quantum Mechanical Algorithm for Database Search,” *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 212–219, 1996. doi: 10.1145/237814.237866; arXiv: quant-ph/9605043.
- [3] C. R. Harris et al., “Array programming with NumPy,” *Nature* **585**, 357–362 (2020). doi: 10.1038/s41586-020-2649-2.
- [4] J. Bradbury et al., “JAX: composable transformations of Python+NumPy programs,” software release, 2018. <https://github.com/google/jax>.
- [5] Arm Ltd., “Arm Neon Intrinsics Reference,” Arm C Language Extensions documentation. https://arm-software.github.io/acle/neon_intrinsics/advsimd.html.